SMK_CASHEW Step-by-Step Project Guide

Backend-first roadmap for your existing SMK_CASHEW project using Node.js, Express, MongoDB Atlas, and a separate frontend and docs folder.

Current status: backend already started, Express/dotenv/cors/nodemon installed, MongoDB cluster created. This guide starts from that point and shows what to do next in order.

1. Final Folder Structure

Your project root should look like this:

```
SMK_CASHEW/
backend/
frontend/
database/
docs/
```

Inside backend, build this structure:

```
backend/
src/
config/
db.js
controllers/
productController.js
authController.js
cartController.js
middleware/
authMiddleware.js
errorMiddleware.js
validateMiddleware.js
models/
User.js
Product.js
Cart.js
Order.js
routes/
productRoutes.js
authRoutes.js
cartRoutes.js
seed/
productSeeder.js
server.js
.env
.env.example
package.json
```

2. Backend Setup

Go to the backend folder and make sure the backend package is initialized.

```
cd SMK_CASHEW/backend
npm init -y
```

Install the important backend dependencies. You already installed some; running this command again is safe because npm will keep existing packages.

```
npm install express mongoose dotenv cors bcryptjs jsonwebtoken cookie-parser
express-validator helmet express-rate-limit morgan
```

```
npm install -D nodemon
```

Update backend/package.json scripts:

```
"scripts": {
"dev": "nodemon src/server.js",
"start": "node src/server.js",
"seed": "node src/seed/productSeeder.js"
}
```

3. Environment Variables

Create backend/.env with your actual MongoDB Atlas connection string. Never commit this file.

```
PORT=5000
MONGO_URI=mongodb+srv://YOUR_USER:YOUR_PASSWORD@YOUR_CLUSTER.mongodb.net/smk_cashew
JWT_SECRET=make_this_a_long_random_secret
CLIENT_URL=http://localhost:5173
NODE_ENV=development
```

Create backend/.env.example with safe placeholder values so other developers know what keys are needed.

4. MongoDB Connection

Create backend/src/config/db.js:

```
const mongoose = require("mongoose");

const connectDB = async () => {
try {
await mongoose.connect(process.env.MONGO_URI);
console.log("MongoDB connected");
} catch (error) {
console.error("MongoDB connection failed:", error.message);
process.exit(1);
}
};

module.exports = connectDB;
```

5. Express Server

Create backend/src/server.js. Start with health check, security middleware, JSON parsing, CORS, and route registration.

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const helmet = require("helmet");
const morgan = require("morgan");
const rateLimit = require("express-rate-limit");
const connectDB = require("./config/db");

const app = express();
connectDB();

app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(express.json());
app.use(morgan("dev"));
```

```
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));

app.get("/api/health", (req, res) => {
res.json({ status: "ok", service: "SMK_CASHEW API" });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Run and test:

```
npm run dev
# Open http://localhost:5000/api/health
```

6. Product Model

Create backend/src/models/Product.js:

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
name: { type: String, required: true, trim: true },
slug: { type: String, required: true, unique: true, lowercase: true },
type: { type: String, enum: ["raw", "roasted", "salted", "flavored", "organic"], required:
true },
grade: { type: String, required: true },
weightGrams: { type: Number, required: true },
price: { type: Number, required: true, min: 0 },
stock: { type: Number, required: true, min: 0 },
images: [{ type: String }],
description: { type: String, required: true },
isActive: { type: Boolean, default: true }
}, { timestamps: true });

module.exports = mongoose.model("Product", productSchema);
```

7. Product API

Create backend/src/controllers/productController.js:

```
const Product = require("../models/Product");

exports.getProducts = async (req, res, next) => {
try {
const { type, minPrice, maxPrice, page = 1, limit = 12 } = req.query;
const filter = { isActive: true };
if (type) filter.type = type;
if (minPrice || maxPrice) filter.price = {};
if (minPrice) filter.price.$gte = Number(minPrice);
if (maxPrice) filter.price.$lte = Number(maxPrice);

const skip = (Number(page) - 1) * Number(limit);
const products = await Product.find(filter).sort({ createdAt: -1
}).skip(skip).limit(Number(limit));
const total = await Product.countDocuments(filter);
res.json({ products, total, page: Number(page), pages: Math.ceil(total / Number(limit)) });
} catch (error) { next(error); }
};

exports.getProductById = async (req, res, next) => {
try {
```

```
const product = await Product.findById(req.params.id);
if (!product || !product.isActive) return res.status(404).json({ message: "Product not found"
});
res.json(product);
} catch (error) { next(error); }
};
```

Create backend/src/routes/productRoutes.js:

```
const express = require("express");
const { getProducts, getProductById } = require("../controllers/productController");
const router = express.Router();

router.get("/", getProducts);
router.get("/:id", getProductById);

module.exports = router;
```

Register the route in server.js before app.listen:

```
app.use("/api/products", require("./routes/productRoutes"));
```

8. Product Seeder

Create backend/src/seed/productSeeder.js and seed at least 10 products before starting the frontend.

```
require("dotenv").config();
const connectDB = require("../config/db");
const Product = require("../models/Product");

const products = [
{ name: "Raw Whole Cashews W240", slug: "raw-whole-cashews-w240", type: "raw", grade: "W240",
weightGrams: 500, price: 649, stock: 50, images: [], description: "Premium whole raw cashews
with rich creamy texture." },
{ name: "Raw Whole Cashews W320", slug: "raw-whole-cashews-w320", type: "raw", grade: "W320",
weightGrams: 500, price: 549, stock: 60, images: [], description: "Everyday premium raw
cashews for snacking and cooking." },
{ name: "Roasted Salted Cashews", slug: "roasted-salted-cashews", type: "salted", grade:
"W320", weightGrams: 250, price: 329, stock: 80, images: [], description: "Crunchy roasted
cashews with balanced salt." },
{ name: "Masala Cashews", slug: "masala-cashews", type: "flavored", grade: "W320",
weightGrams: 250, price: 349, stock: 45, images: [], description: "Spiced cashews with Indian
masala seasoning." },
{ name: "Organic Cashews", slug: "organic-cashews", type: "organic", grade: "W240",
weightGrams: 500, price: 749, stock: 30, images: [], description: "Organic cashews sourced
for premium quality." }
];

const seed = async () => {
await connectDB();
await Product.deleteMany();
await Product.insertMany(products);
console.log("Products seeded");
process.exit();
};

seed();
```

Run:

```
npm run seed
# Then test GET http://localhost:5000/api/products
```

9. Authentication

After products work, build auth. Create User model, auth controller, routes, and JWT middleware.

- POST /api/auth/register: create user with hashed password
- POST /api/auth/login: verify password and return token
- GET /api/auth/me: return current logged-in user

Use bcryptjs for hashing and jsonwebtoken for token creation.

10. Cart

After auth works, build cart endpoints. Keep cart server-side for logged-in users and localStorage later for guests on frontend.

- GET /api/cart: get current user's cart
- POST /api/cart: add item
- PUT /api/cart/:productId: update quantity
- DELETE /api/cart/:productId: remove item

11. Frontend Starts Only After Backend Products Work

Once GET /api/products returns real data, scaffold frontend:

```
cd SMK_CASHEW
npm create vite@latest frontend
# Choose React
cd frontend
npm install axios react-router-dom lucide-react
```

Frontend order:

- Navbar
- Home page
- Product listing page
- Product detail page
- Cart context and cart page
- Login and register pages

12. Immediate Checklist

Order	Task	Success check
1	Confirm backend structure	src/config, controllers, models, routes exist
2	Add .env	MongoDB URI is loaded
3	Create db.js	MongoDB connected appears
4	Create server.js	/api/health returns ok
5	Create Product model	No schema errors
6	Create product routes	/api/products works

7	Seed products	Products visible in API response
8	Start auth	Register/login works
9	Start cart	Logged-in cart works
10	Then frontend	Product list consumes backend API

13. Do Not Skip These Rules

- Do not hard-code your MongoDB password in code. Keep it only in .env.
- Do not start payment integration before cart and order creation work.
- Do not start admin dashboard before product browsing and cart are stable.
- Keep docs/api-spec.md updated while you build endpoints.
- Commit after each working milestone: health route, products route, auth, cart, frontend scaffold.